UNIT-1

CHAPTER-2: INTELLIGENT AGENTS

2.1 Agents and Environments

- **Agent:** An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.
- A human agent has eyes, ears, and other organs for sensors and hands, legs, vocal tract, and so on for actuators.
- Robotic agent might have cameras and infrared range finders for sensors and various motors for actuators.

- A software agent receives keystrokes, file contents, and network packets as sensory inputs and acts on the environment by displaying on the screen, writing files, and sending network packets.
- **PERCEPT:** Refer to the agent's perceptual inputs at any given instant.
- **PERCEPT SEQUENCE:** It is the complete history of everything the agent has ever perceived
- In general, an agent's choice of action at any given instant can depend on the entire percept sequence observed to date, but not on anything it hasn't perceived.

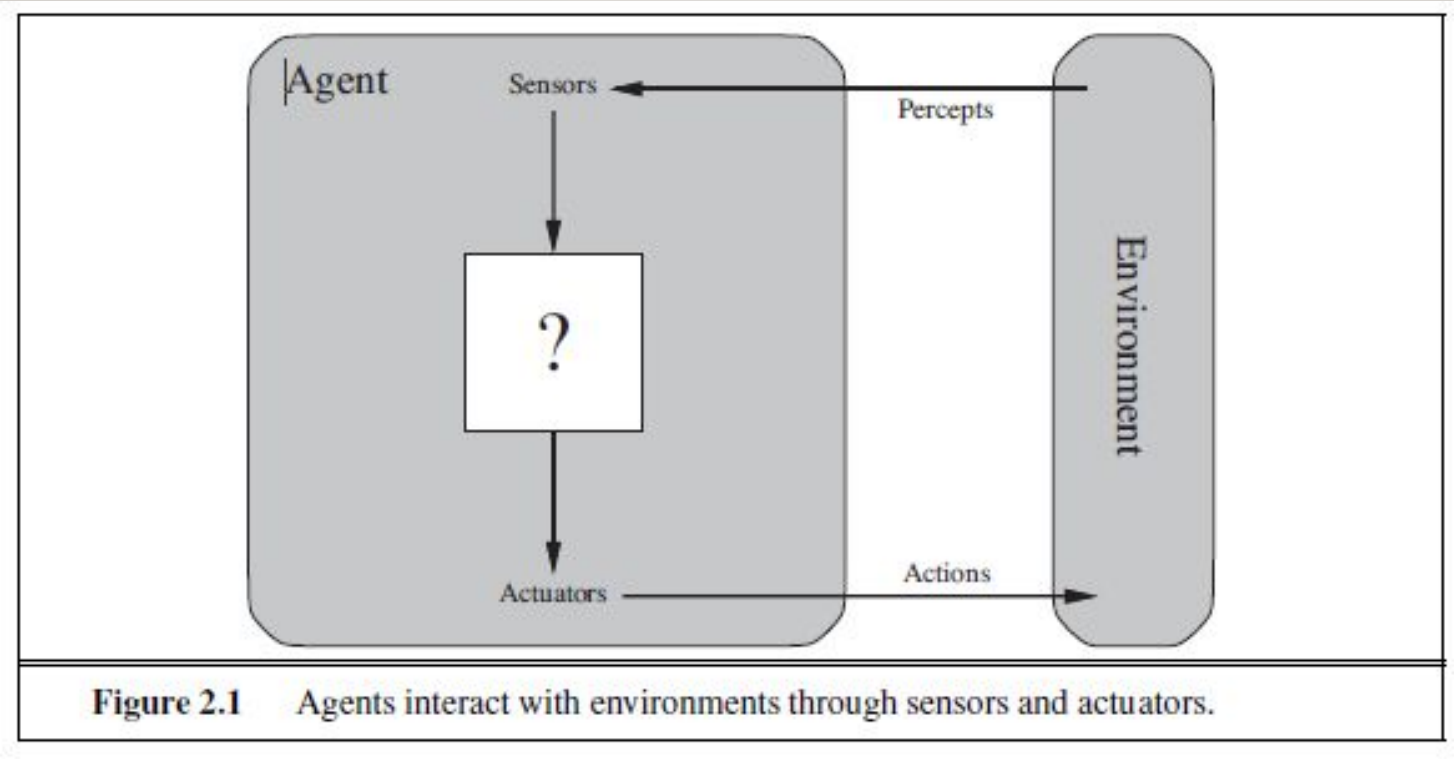


Figure 2.1 Agents interact with environments through sensors and actuators.

- Mathematically speaking, we say that an agent's behavior is described by the **agent function** that maps any given percept sequence to an action.

- We can imagine tabulating the agent function that describes any given agent; for most agents, this would be a very large table—infinite, in fact, unless we place a bound on the length of percept sequences we want to consider.
- Given an agent to experiment with, we can, in principle, construct this table by trying out all possible percept sequences and recording which actions the agent does in response.1 The table is, of course, an external characterization of the agent.

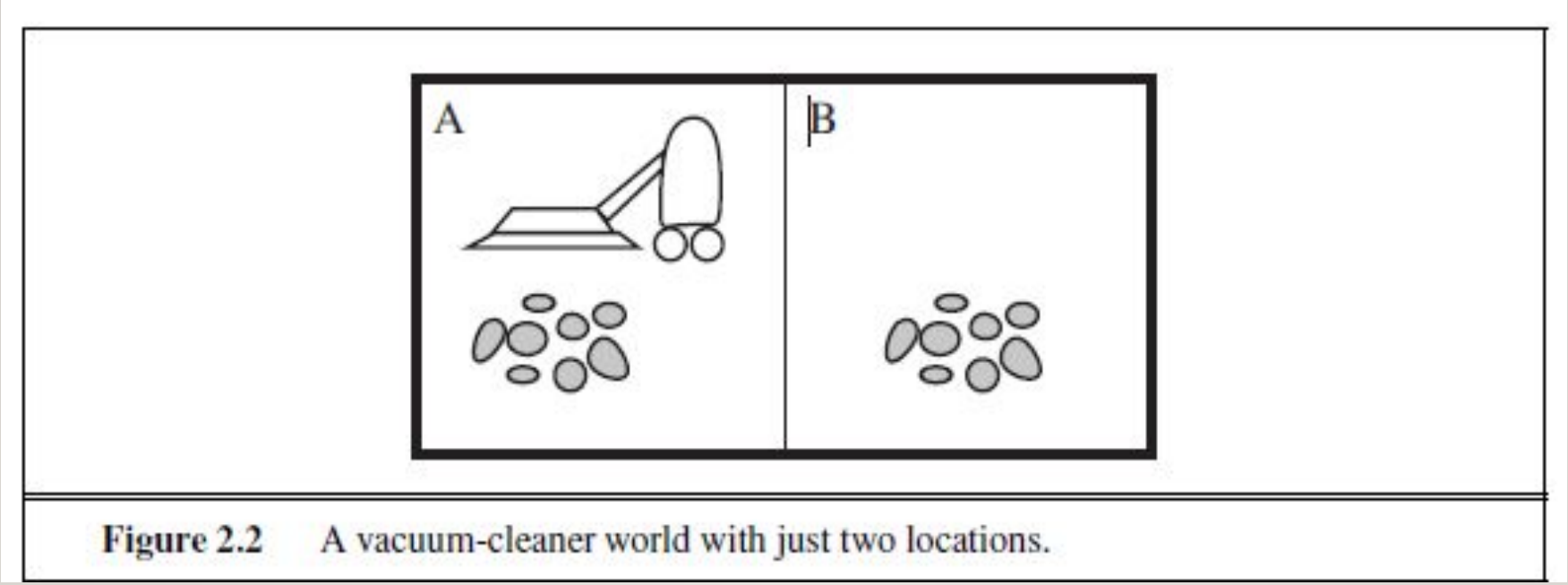


Figure 2.2 A vacuum-cleaner world with just two locations.

- To illustrate these ideas, we use a very simple example—the vacuum-cleaner world. This world is so simple that we can describe everything that happens; it's also a made-up world, so we can invent many variations.

- This particular world has just two locations: squares A and B. The vacuum agent perceives which square it is in and whether there is dirt in the square. It can choose to move left, move right, suck up the dirt, or do nothing.
- One very simple agent function is the following: if the current square is dirty, then suck; otherwise, move to the other square.
- A partial tabulation of this agent function is shown in Figure 2.3 and an agent program that implements it

Percept sequence	Action
$[A, Clean]$	*Right*
$[A, Dirty]$	*Suck*
$[B, Clean]$	*Left*
$[B, Dirty]$	*Suck*
$[A, Clean], [A, Clean]$	*Right*
$[A, Clean], [A, Dirty]$	*Suck*
⋮	⋮
$[A, Clean], [A, Clean], [A, Clean]$	*Right*
$[A, Clean], [A, Clean], [A, Dirty]$	*Suck*
⋮	⋮

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

- we see that various vacuum-world agents can be defined simply by filling in the right-hand column in various ways. The obvious question, then, is this: What is the right way to fill out the table? In other words, what makes an agent good or bad, intelligent or stupid? We answer these questions in the next section.

- This particular world has just two locations: squares A and B. The vacuum agent perceives which square it is in and whether there is dirt in the square. It can choose to move left, move right, suck up the dirt, or do nothing.
- One very simple agent function is the following: if the current square is dirty, then suck; otherwise, move to the other square.
- A partial tabulation of this agent function is shown in Figure 2.3 and an agent program that implements it.
- One could view a hand-held calculator as an agent that chooses the action of displaying “4” when given the percept sequence “2 + 2 =,” but such an analysis would hardly aid our understanding of the calculator.

2.2 Good behavior: The concept of rationality

- When an agent is plunked down in an environment, it generates a sequence of actions according to the percepts it receives. This sequence of actions causes the environment to go through a sequence of states.
- If the sequence is desirable, then the agent has performed well. This notion of desirability is captured by a performance measure that evaluates any given sequence of environment states.

- Consider, for example, the vacuum-cleaner agent from the preceding section. We might propose to measure performance by the amount of dirt cleaned up in a single eight-hour shift.
- With a rational agent, of course, what you ask for is what you get. A rational agent can maximize this performance measure by cleaning up the dirt, then dumping it all on the floor, then cleaning it up again, and so on. A more suitable performance measure would reward the agent for having a clean floor.
- A **rational agent** does exactly what it's told *based on its performance measure*, not necessarily what humans *intend*. So, the performance measure must be carefully designed to reflect **true goals** (e.g., maintaining cleanliness over time, minimizing power use and noise), not just surface-level actions.

2.2.1 Rationality

 What is rational at any given time depends on four things:

- The performance measure that defines the criterion of success.
- The agent's prior knowledge of the environment.
- The actions that the agent can perform.
- The agent's percept sequence to date.

- First, we need to say what the performance measure is, what is known about the environment, and what sensors and actuators the agent has.
- Let us assume the following:

• The performance measure awards one point for each clean square at each time step, over a "lifetime" of 1000 time steps.

• The "geography" of the environment is known a priori (Figure 2.2) but the dirt distribution and the initial location of the agent are not. Clean squares stay clean and sucking cleans the current square. The Left and Right actions move the agent left and right except when this would take the agent outside the environment, in which case the agent remains where it is.

- The only available actions are Left , Right, and Suck.
- The agent correctly perceives its location and whether that location contains dirt.

For example, once all the dirt is cleaned up, the agent will oscillate needlessly back and forth; if the performance measure includes a penalty of one point for each movement left or right, the agent will fare poorly. A better agent for this case would do nothing once it is sure that all the squares are clean. If clean squares can become dirty again, the agent should occasionally check and re-clean them if needed. If the geography of the environment is unknown, the agent will need to explore it rather than stick to squares A and B.

2.2.2 Omniscience, learning, and autonomy

- We need to be careful to distinguish between rationality and omniscience. An omniscient agent knows the actual outcome of its actions and can act accordingly; but omniscience is impossible in reality.
- **Example:**
- You're walking on the Champs Élysées and see your friend across the street. There's no traffic, and it's safe, so you start to cross.
- Suddenly, a cargo door from a plane falls and hits you.
- Question: Were you irrational?
- Answer: No! You made the best decision based on what you knew.

Concept	Explanation
Rationality	Doing the best you can with the **information available**.
Perfection	Doing what turns out to be best **after everything happens**.

- A rational agent doesn't have to know everything (omniscience). It just needs to make the best decision based on what it has seen (its percepts) so far
- **Example:** Crossing the Road Imagine an agent (like a robot or person):
- It doesn't look both ways before crossing.
- So it doesn't know a truck is coming.
- Should it still cross because its "view" says nothing is wrong?
- No! That would still be irrational.

- Why?
  - Risk is high — Crossing without checking is dangerous.
  - A rational agent should first look (gather info) to improve its future decisions.
  - This action of gathering useful info is called information gathering.

Another Example: Vacuum Cleaner

In an unknown house, a vacuum agent: Should explore rooms first (gather info). So it knows where dirt is and what path to follow

Rational agents don't just act—they also collect info first to make smarter decisions.

Rational Agents Must Learn from Experience

A good agent doesn't just act—it also:

- Gathers information
- Learns from what it sees
- Updates its behavior if things change

- An **autonomous agent** is one that makes decisions based on its own observations and experiences, rather than just relying on instructions given by its designer.
- A rational agent should be autonomous so it can learn from the environment and adapt when the initial knowledge it was given is incomplete or incorrect.
- For example, a vacuum-cleaning agent that learns where dirt is likely to appear will perform better than one that blindly follows fixed rules.
- However, full autonomy is not needed right from the beginning—an agent with no experience would act randomly without some basic built-in knowledge, just like animals are born with instincts to help them survive until they learn.

2.3 The Nature Of Environments

- **Task environments** are essentially the "problems" to which rational agents are the "solutions."

2.3.1 Specifying the task environment

- The rationality of the simple vacuum-cleaner agent, we had to specify the performance measure, the environment, and the agent's actuators and sensors. PEAS (Performance, Environment, Actuators, Sensors)

Example: automated taxi driver.

We should point out, before the reader becomes alarmed, that a fully automated taxi is currently somewhat beyond the capabilities of existing technology.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Figure 2.4 PEAS description of the task environment for an automated taxi.

- First, what is the performance measure to which we would like our automated driver to aspire?
- Desirable qualities include
  - getting to the correct destination;
  - minimizing fuel consumption and wear and tear;
  - minimizing the trip time or cost;
  - minimizing violations of traffic laws and disturbances to other drivers;
  - maximizing safety and passenger comfort;
  - Maximizing profits.

- Next, what is the driving environment that the taxi will face?
- Any taxi driver must deal with
  - a variety of roads,
  - ranging from rural lanes and urban alleys to 12-lane freeways.
  - The roads contain other traffic, pedestrians, stray animals, road works, police cars, puddles and potholes.

- The **actuators** for an automated taxi include those available to a human driver: control over the engine through the accelerator and control over steering and braking.
- In addition, it will need output to a display screen or voice synthesizer to talk back to the passengers, and perhaps some way to communicate with other vehicles, politely or otherwise.
- The basic **sensors** for the taxi will include one or more controllable video cameras so that it can see the road; it might augment these with infrared or sonar sensors to detect distances to other cars and obstacles.

- To avoid speeding tickets, the taxi should have a speedometer, and to control the vehicle properly, especially on curves, it should have an accelerometer.
- **software agents** (or software robots or softbots) exist in rich, unlimited domains. Imagine a softbot Web site operator designed to scan Internet news sources and show the interesting items to its users, while selling advertising space to generate revenue.
- To do well, that operator will need some natural language processing abilities, it will need to learn what each user and advertiser is interested in, and it will need to change its plans dynamically—for example, when the connection for one news source goes down or when a new one comes online.

2.3.2 Properties of task environments

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Figure 2.5 Examples of agent types and their PEAS descriptions.

Fully observable vs. partially observable environment:

- If an agent's sensors give it access to the complete state of the environment at each point in time, then we say that the task environment is fully observable.
- Fully observable environments are convenient because the agent need not maintain any internal state to keep track of the world.
- An environment might be partially observable because of noisy and inaccurate sensors or because parts of the state are simply missing from the sensor data
- **Example:** A vacuum agent with only a local dirt sensor cannot tell whether there is dirt in other squares, and an automated taxi cannot see what other drivers are thinking. If the agent has no sensors at all then the environment is unobservable

Single agent vs. multiagent environment:

- The distinction between single-agent and multiagent environments may seem simple enough. For example, an agent solving a crossword puzzle by itself is clearly in a single-agent environment, whereas an agent playing chess is in a two agent environment.

- For example, in chess, the opponent entity B is trying to maximize its performance measure, which, by the rules of chess, minimizes agent A's performance measure. Thus, chess is a **competitive** multiagent environment. In the taxi-driving environment, on the other hand, avoiding collisions maximizes the performance measure of all agents, so it is a partially **cooperative** multiagent environment.

Deterministic vs. stochastic environment:

- If the next state of the environment is completely determined by the current state and the action executed by the agent, then we say the environment is deterministic; otherwise, it is stochastic.
- **Example for Deterministic:** The vacuum world as we described it is deterministic, but variations can include stochastic elements such as randomly appearing dirt and an unreliable suction mechanism
- **Example for Stochastic:** Taxi driving is clearly stochastic in this sense, because one can never predict the behavior of traffic exactly.

- **uncertain** if it is not fully observable or not deterministic. One final note: our use of the word "stochastic" generally implies that uncertainty about outcomes is quantified in terms of probabilities;
- **Nondeterministic** environment is one in which actions are characterized by their possible outcomes, but no probabilities are attached to them.

Episodic vs. sequential environment:

- In an **episodic task environment**, the agent's experience is divided into atomic episodes. In each episode the agent receives a percept and then performs a single action. Crucially, the next episode does not depend on the actions taken in previous episodes.

- For example, an agent that has to spot defective parts on an assembly line bases each decision on the current part, regardless of previous decisions; moreover, the current decision doesn't affect whether the next part is defective.
- In **sequential environments**, on the other hand, the current decision could affect all future decisions.
- Chess and taxi driving are sequential: in both cases, short-term actions can have long-term consequences.

Static vs. dynamic environment :

- If the environment can change while an agent is deliberating, then we say the environment is dynamic for that agent; otherwise, it is static
- Static environments are easy to deal with because the agent need not keep looking at the world while it is deciding on an action, nor need it worry about the passage of time.
- Dynamic environments, on the other hand, are continuously asking the agent what it wants to do; if it hasn't decided yet, that counts as deciding to do nothing.

- If the environment itself does not change with the passage of time but the agent's performance score does, then we say the environment is semidynamic.
- Taxi driving is clearly dynamic: the other cars and the taxi itself keep moving while the driving algorithm dithers about what to do next. Chess, when played with a clock, is semidynamic. Crossword puzzles are static.

Discrete vs. continuous environment:

- The discrete/continuous distinction applies to the state of the environment, to the way time is handled, and to the percepts and actions of the agent.
- For example, the chess environment has a finite number of distinct states (excluding the clock).

- Chess also has a discrete set of percepts and actions. Taxi driving is a continuous-state and continuous-time problem: the speed and location of the taxi and of the other vehicles sweep through a range of continuous values and do so smoothly over time.

- **Known vs. unknown environment:**

- Strictly speaking, this distinction refers not to the environment itself but to the agent's (or designer's) state of knowledge about the "laws of physics" of the environment.

- In a known environment, the outcomes (or outcome probabilities if the environment is stochastic) for all actions are given. Obviously, if the environment is unknown, the agent will have to learn how it works in order to make good decisions.

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.

2.4 The structure of agents

- The job of AI is to design an agent program that implements the agent function— the mapping from percepts to actions. We assume this program will run on some sort of computing device with physical sensors and actuators—we call this the architecture:

  agent = architecture + program

- **Architecture:** Architecture is machinery that an AI agent executes on.
- **Agent Function:** It is used to map a percept to an action
- **Agent Program:** Agent Program is an implementation of agent function, An agent program executes on the physical architecture to produce function.

If the program is going to recommend actions like Walk, the architecture had better have legs. The architecture might be just an ordinary PC, or it might be a robotic car with several onboard computers, cameras, and other sensors

2.4.1 Agent Programs

- Agent programs take the current percept as input and return an action. In contrast, agent functions use the entire percept history. Since the environment gives only the current percept, the agent must store past percepts if needed.
- Actions are chosen by looking up a predefined table based on percept sequences.

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
              table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

- Agents can be grouped into five classes based on their degree of perceived intelligence and capability. All these agents can improve their performance and generate action over the time.
  - Simple reflex agents;
  - Model-based reflex agents;
  - Goal-based agents; and
  - Utility-based agents.

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```

Figure 2.8 The agent program for a simple reflex agent in the two-state vacuum environment. This program implements the agent function tabulated in Figure 2.3.

2.4.2 Simple reflex agents

- These agents select actions on the basis of the current percept, ignoring the rest of the percept history.
- A condition–action rule is a simple rule that tells an agent what to do when a specific condition is true.
- For example, in an automated taxi:
  - Condition: "The car in front is braking."
  - Action: "Initiate braking."
- So, when the agent detects the condition (like brake lights turning on), it automatically performs the matching action (like applying the brakes). This is called a simple reflex behavior because it reacts immediately based on current input, without thinking about the future or the past.

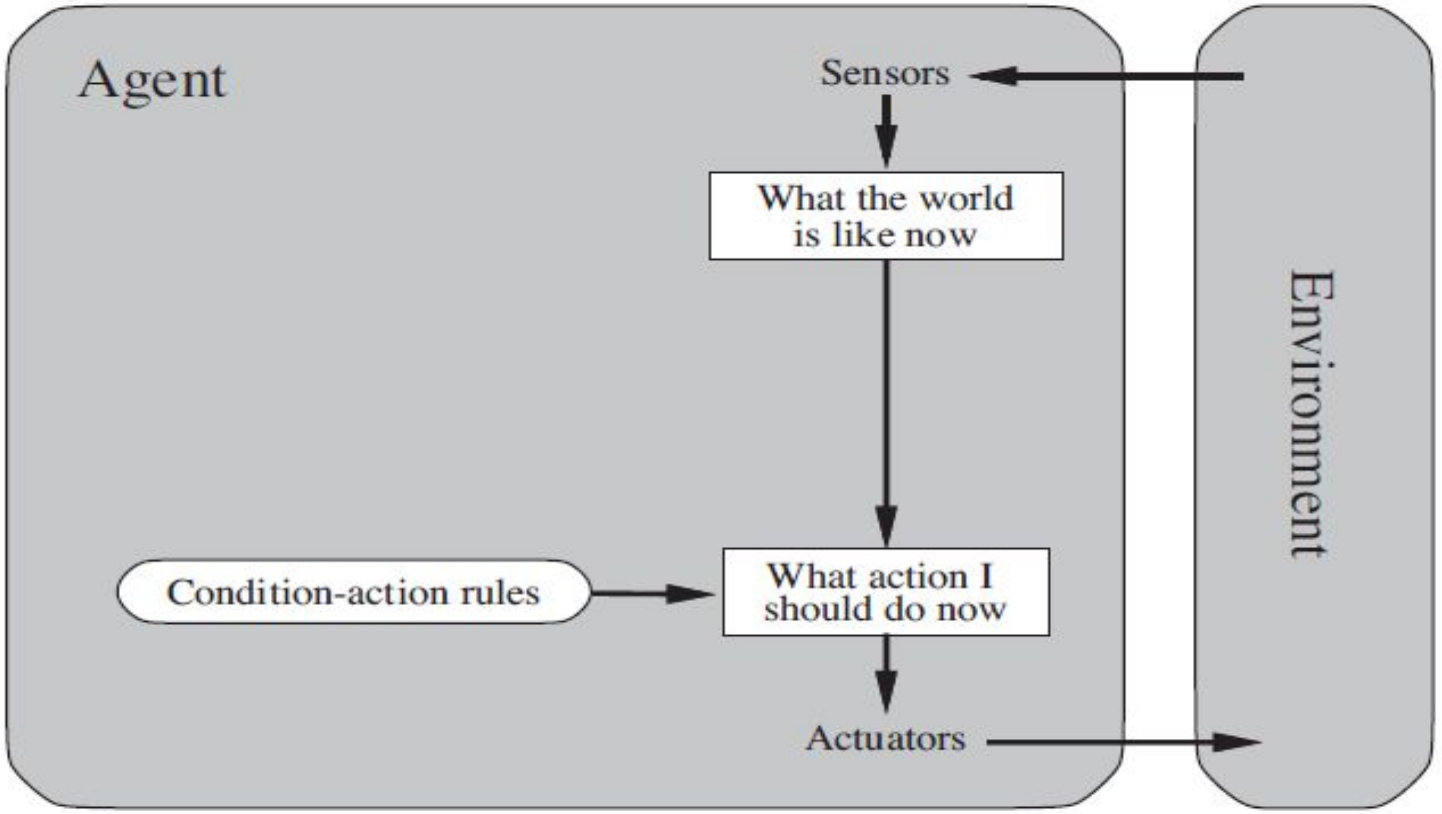


Figure 2.9 Schematic diagram of a simple reflex agent.

We use rectangles to denote the current internal state of the agent's decision process, and ovals to represent the background information used in the process.

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

- The function generates an abstracted description of the current state from the percept, and the function returns the first rule in the set of rules that matches the given state description.

2.4.3 Model-based reflex agents

- The Model-based agent can work in a partially observable environment, and track the situation.
- A model-based agent has two important factors:
  - **Model:** It is knowledge about "how things happen in the world," so it is called a Model-based agent.
  - **Internal State:** It is a representation of the current state based on percept history.
- These agents have the model, "which is knowledge of the world" and based on the model they perform actions.
- Updating the agent state requires information about:
  - How the world evolves
  - How the agent's action affects the world.

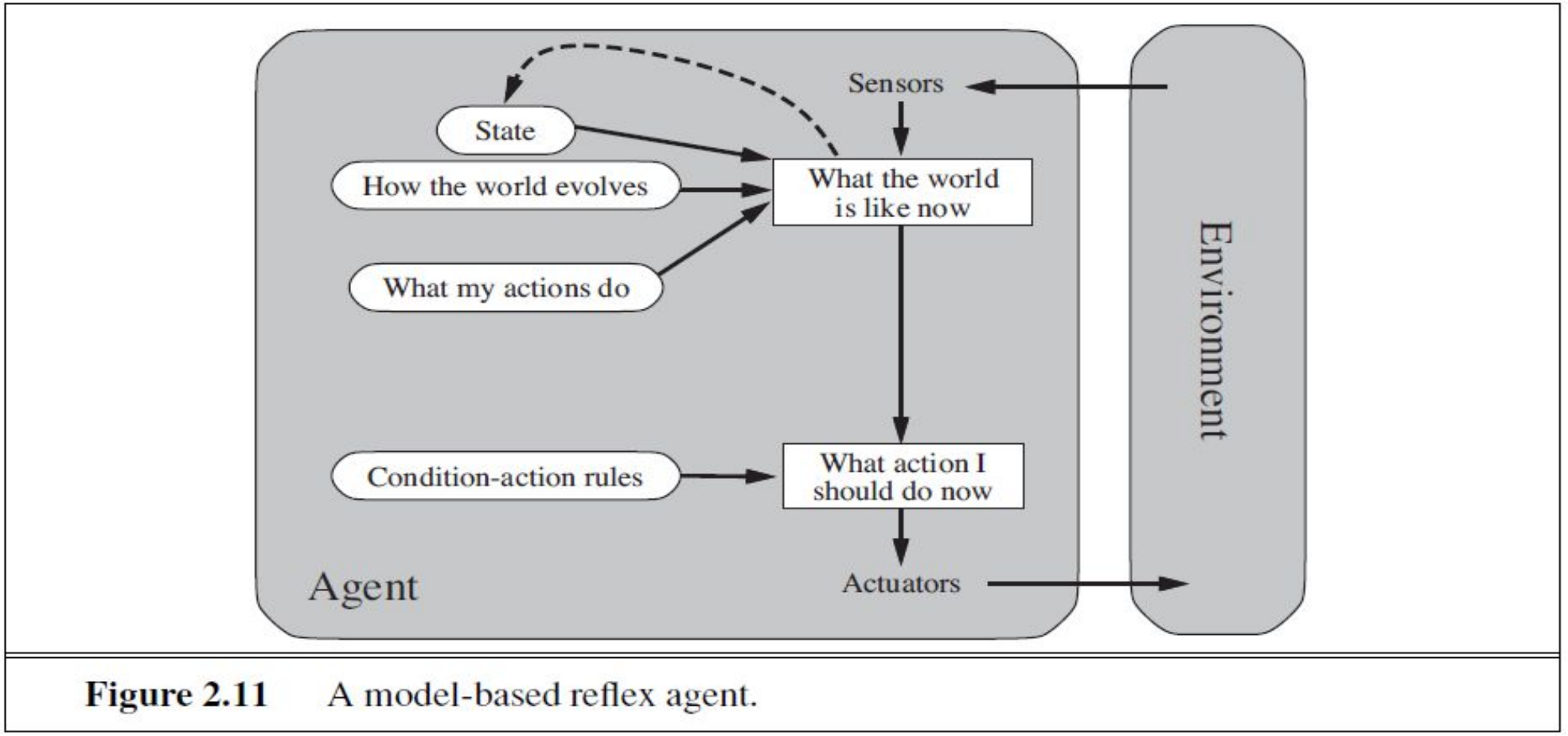


Figure 2.11 A model-based reflex agent.

The agent will determine the current state of a partially observable environment exactly. Instead, the box labeled "what the world is like now"

For example, an automated taxi may not be able to see around the large truck that has stopped in front of it and can only guess about what may be causing the hold-up. Thus, uncertainty about the current state may be unavoidable, but the agent still has to make a decision.

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
              model, a description of how the next state depends on current state and action
              rules, a set of condition–action rules
              action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

2.4.4 Goal-based agents

- The knowledge of the current state environment is not always sufficient to decide for an agent what to do.
- The agent needs to know its goal which describes desirable situations. Goal-based agents expand the capabilities of the model-based agent by having the "goal" information.
- They choose an action, so that they can achieve the goal.
- These agents may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not.
- Such considerations of different scenarios are called searching and planning, which makes an agent proactive.

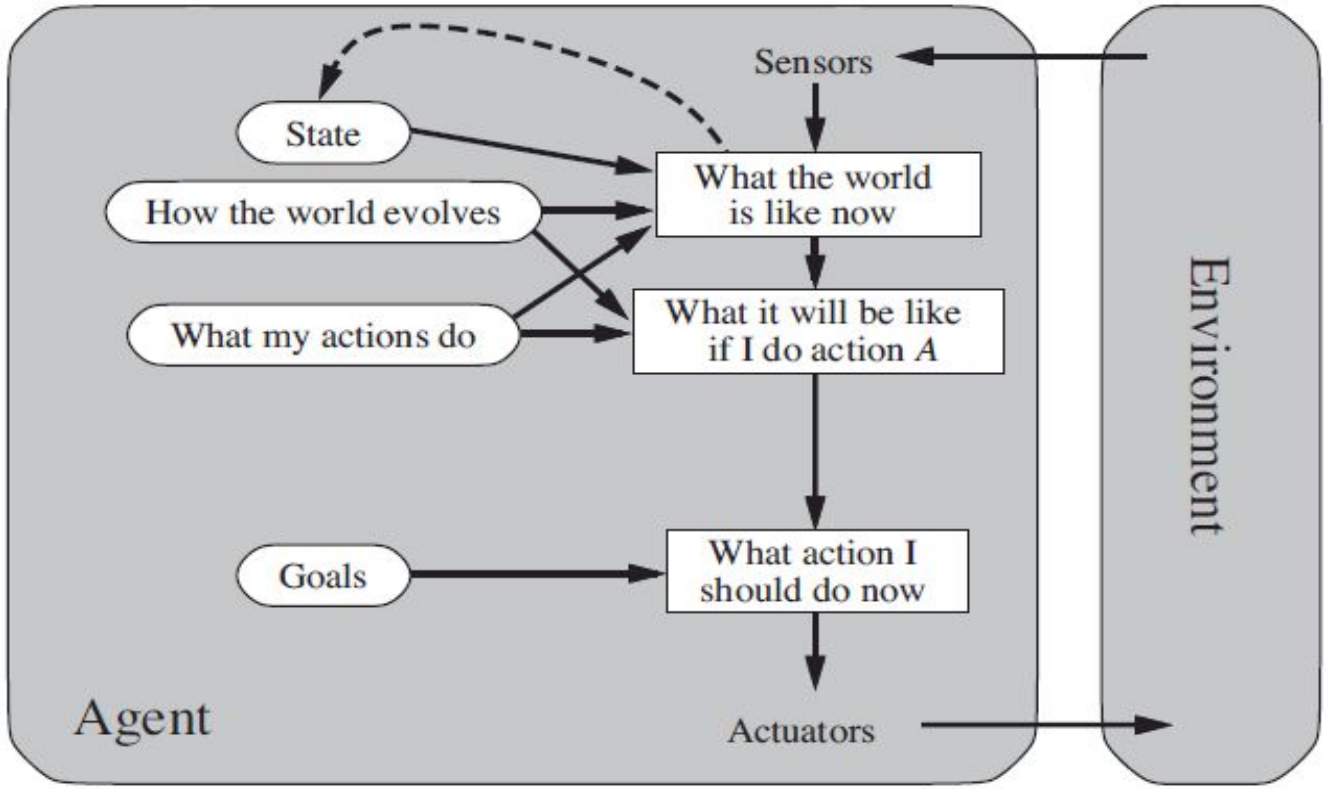


Figure 2.13 A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

2.4.5 Utility-based agents

- These agents are similar to the goal-based agent but provide an extra component of utility measurement which makes them different by providing a measure of success at a given state.
- Utility-based agent acts based not only on goals but also the best way to achieve the goal.
- The Utility-based agent is useful when there are multiple possible alternatives, and an agent has to choose in order to perform the best action.
- The utility function maps each state to a real number to check how efficiently each action achieves the goals.

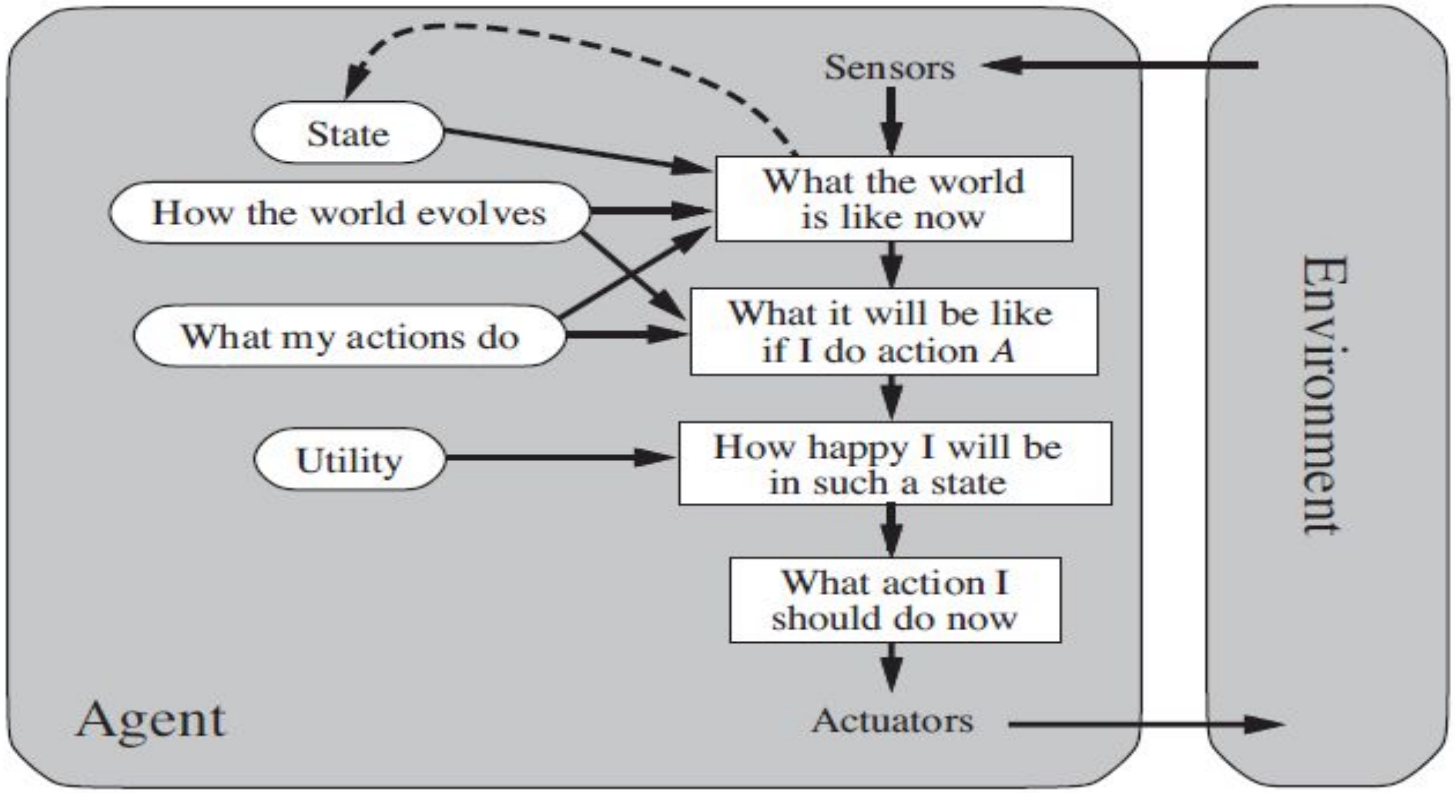


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

2.4.6 Learning agents

- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- It starts to act with basic knowledge and then able to act and adapt automatically through learning.
- A learning agent has mainly four conceptual components, which are:
  - **Learning element**: It is **responsible for making** improvements by learning from environment.
  - **Critic**: Learning element takes **feedback from critic** which describes how well the agent is doing with respect to a fixed performance standard.
  - **Performance element**: It is responsible for **selecting external action**.
  - **Problem generator**: This component is **responsible for suggesting actions that will lead to new and informative experiences**.

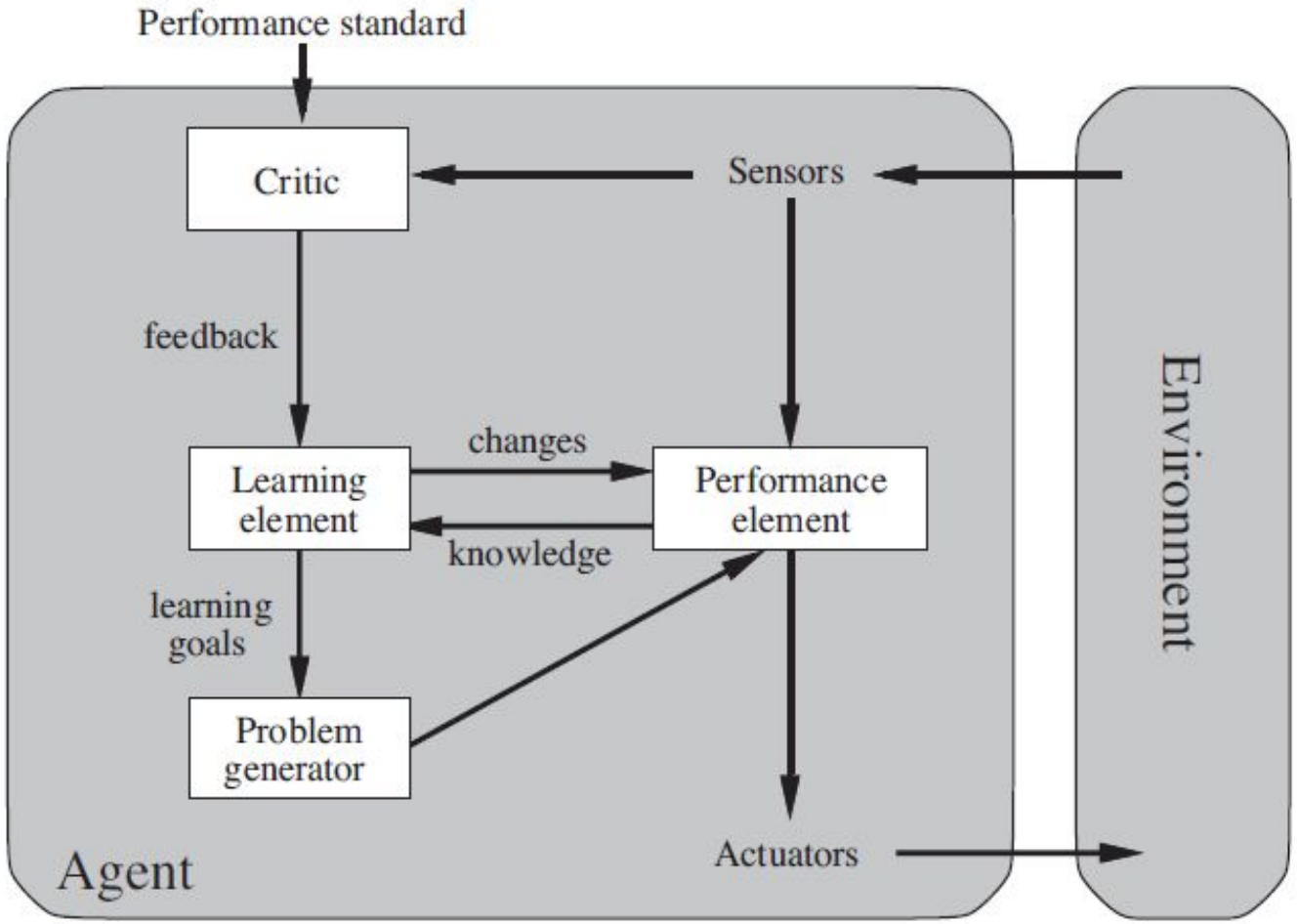


Figure 2.15 A general learning agent.

2.4.7 How the components of agent programs work

- Agent components help answer questions like “What is the world like?” and “What should I do?” using different ways to represent the environment. These representations increase in complexity as **atomic**, **factored**, and **structured**, especially when describing how actions change the world.’
- In an atomic representation, each state of the world is indivisible—it has no internal structure.
- A factored representation splits up each state into a fixed set of variables or attributes, each of which can have a value.

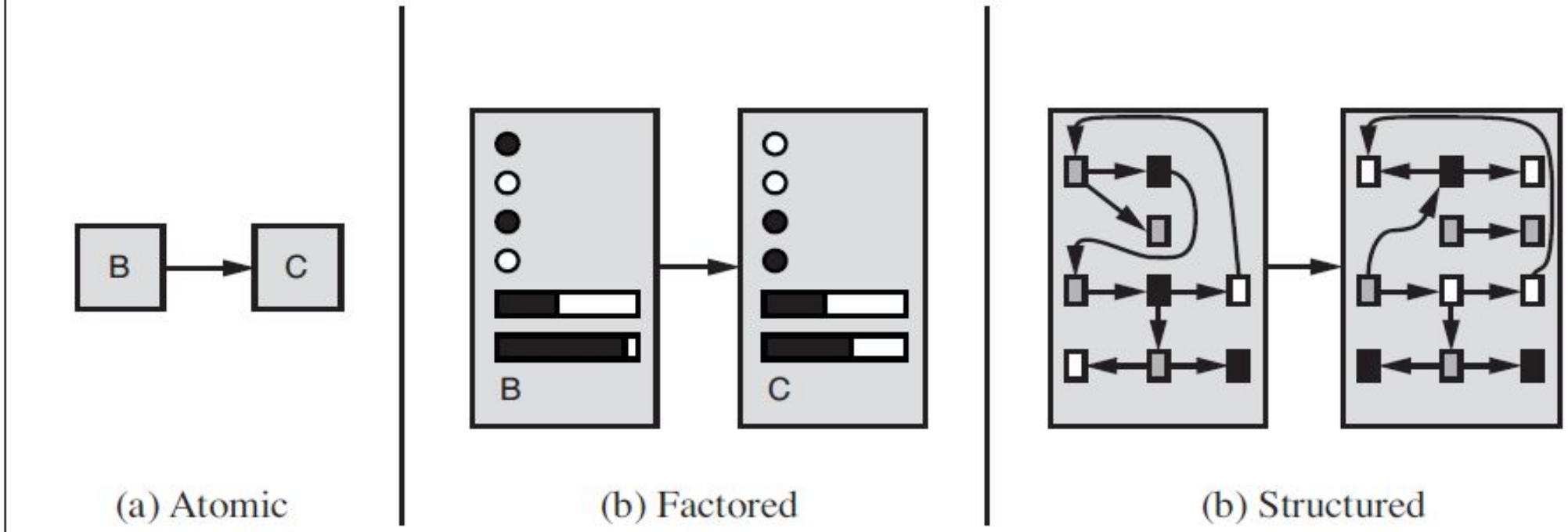


Figure 2.16 Three ways to represent states and the transitions between them. (a) Atomic representation: a state (such as B or C) is a black box with no internal structure; (b) Factored representation: a state consists of a vector of attribute values; values can be Boolean, real-valued, or one of a fixed set of symbols. (c) Structured representation: a state includes objects, each of which may have attributes of its own as well as relationships to other objects.

To understand the world better, we don't just need simple variables like "true" or "false"—we need to know about things (objects) and how they relate to each other. For example, imagine a truck is backing into a driveway, but a cow is standing in the way. It's too complex to represent this situation with just one variable. Instead, we need a structured representation, where we describe objects like the truck, cow, and driveway, and how they interact.

This kind of structured representation is used in:

- Relational databases
- First-order logic
- Probability models
- Knowledge-based learning
- Understanding natural language

2.5 Problem Solving Agents

- Intelligent agents are supposed to maximize their performance measure, achieving this is sometimes simplified if the agent can adopt a goal and aim at satisfying it.
- Imagine an agent on a vacation in Arad, Romania. It wants to do many things—practice Romanian, see tourist spots, enjoy nightlife.
- Now suppose the agent has a nonrefundable flight from Bucharest the next day. In that case, its main goal becomes getting to Bucharest on time. Any plan that doesn't help achieve that can be ignored. This makes decision-making much easier.

- **Goal formulation**, based on the current situation and the agent's performance measure, is the first step in problem solving.
- A goal is a set of world conditions where the agent is successful. The agent's job is to figure out how to act now and later to reach one of these goal states.
- Before doing that, it must decide which actions and situations to think about. If it focused on tiny actions like "move foot an inch" or "turn wheel slightly," it would get stuck due to too many steps and too much uncertainty.
- **Problem formulation** is the process of deciding what actions and states to consider, given a goal.
- An agent in Arad wants to reach Bucharest. There are three roads leading out of Arad—to Sibiu, Timisoara, and Zerind. But the agent doesn't know which way leads to Bucharest unless it knows Romania's geography.

- If the agent has no information about where the roads go, it must guess and try randomly. However, if the agent has a map, it can plan a route by checking which cities lead to Bucharest.
- The map helps the agent understand:
  - What options (cities) are available
  - What actions it can take
  - What states it can reach
- The agent can use this to plan a journey step by step. For example, if the agent finds a path like Arad → Sibiu → Fagaras → Bucharest, it can follow that route to reach the goal.

- We make some assumptions:
  - Observable: The agent knows its current city (e.g., city signs).
  - Discrete: Limited number of roads from each city.
  - Known: The map shows which city each road leads to.
  - Deterministic: If the agent takes a road from Arad to Sibiu, it will reach Sibiu for sure.
- Under these conditions, the solution is just a fixed list of actions (like directions), because the agent knows exactly what will happen. The process SEARCH of looking for a sequence of actions that reaches the goal is called **search.**

- A search algorithm takes a problem as input and returns a list of steps (a **solution**) in the form of action sequence. Once found, the agent follows these steps—this is the execution phase.
- Formulate Goal → Search → Execute Plan. This forms a simple loop.
- While following the plan, the agent doesn't need to react to new information (like an open-loop system), because it already knows what will happen next.

3.1.1 Well-defined problems and solutions:

- A problem can be defined formally by five components:
- The **initial state** that the agent starts in. For example, the initial state for our agent in Romania might be described as In(Arad).

Actions: The agent must know what actions it can take in any given state. Example: From Arad, the agent can go to Sibiu, Timisoara, or Zerind. So, ACTIONS(In(Arad)) = {Go(Sibiu), Go(Timisoara), Go(Zerind)}

Transition Model (Result):

This tells the agent what happens when it takes an action in a state.

RESULT(In(Arad),Go(Zerind)) = In(Zerind)

State Space

The full set of all places the agent can reach from the starting point using any sequence of actions.

It can be seen as a **graph**, where:

- **Nodes** = cities (states)
- **Edges** = roads (actions)

Goal Test

This checks if the current state is the goal.

Example: Is the agent in Bucharest? If yes, the goal is reached.

Goal = {In(Bucharest)}

Path Cost

This is the total cost (like distance or time) to get from the start to the goal.

The agent adds up the **step costs** for each action taken.

Example: Going from Arad to Sibiu costs 140km, So c(Arad, Go(sibiu), Sibiu)=140

A solution to a problem is an action sequence that leads from the initial state to a goal state. Solution quality is measured by the path cost function, and an optimal solution has the lowest path cost among all solutions.

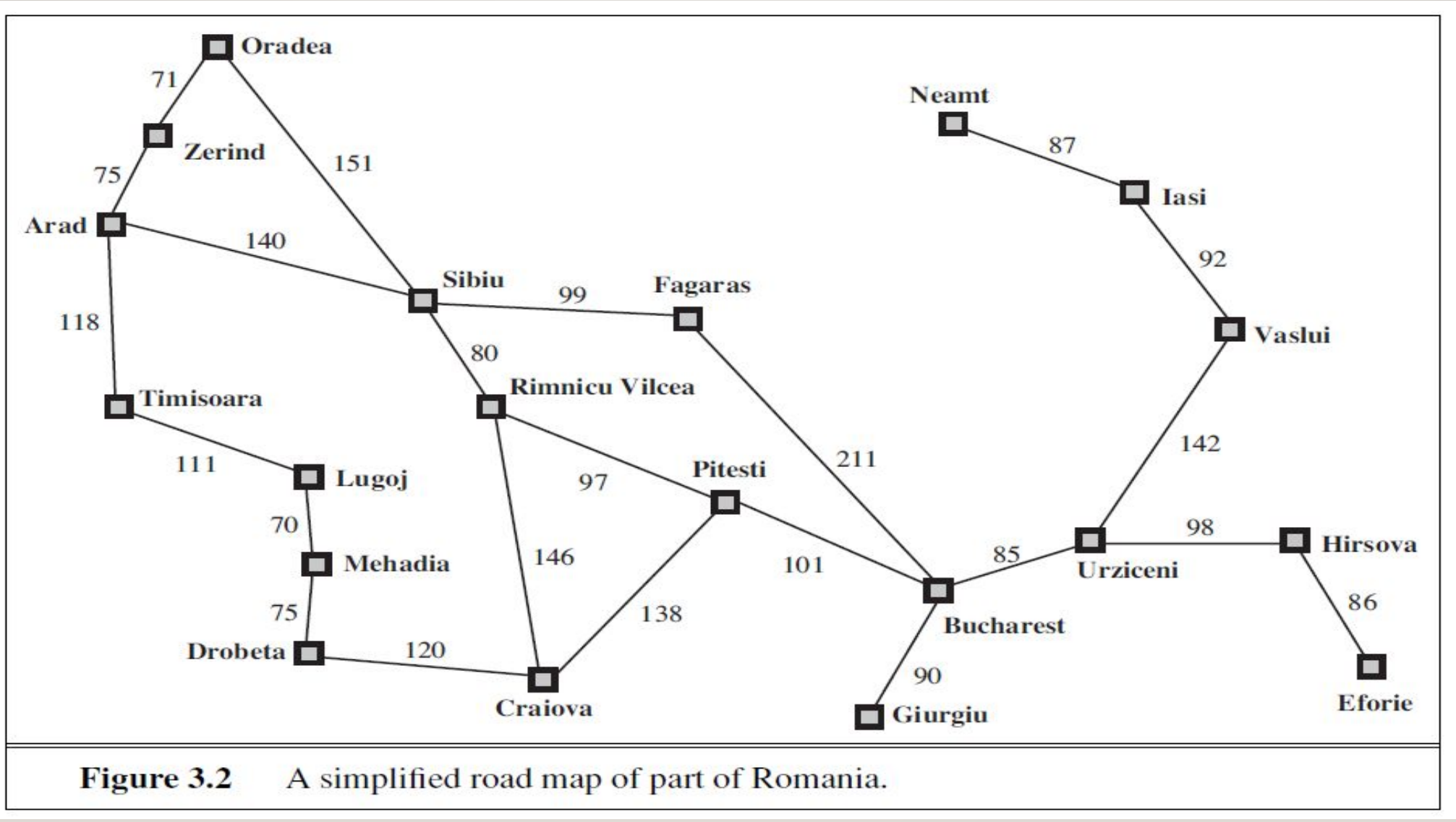


Figure 3.2 A simplified road map of part of Romania.

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
              state, some description of the current world state
              goal, a goal, initially null
              problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
      goal ← FORMULATE-GOAL(state)
      problem ← FORMULATE-PROBLEM(state, goal)
      seq ← SEARCH(problem)
      if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

Figure 3.1 A simple problem-solving agent. It first formulates a goal and a problem, searches for a sequence of actions that would solve the problem, and then executes the actions one at a time. When this is complete, it formulates another goal and starts over.

3.1.2 Formulating problems

When we define the problem of getting to Bucharest, we use a model with simple parts:

- Starting point (e.g., **In(Arad)**)
- Possible actions (e.g., **Go(Sibiu)**)
- Goal test (e.g., **In(Bucharest)**)
- Transition model and path cost

But this is just an **abstract model**—not the full reality.

Real-World vs. Model:

In a real trip, many more details exist:

- Companions, music, scenery, police presence
- Road conditions, weather, fuel, etc.

All of this is ignored in the model because it's not relevant to finding the route to Bucharest.

Abstraction: Abstraction means removing unnecessary details from the problem to make it manageable.

- We ignore small things like "turn the steering wheel slightly" or "turn on the radio."
- We only care about high-level actions like "drive from Arad to Sibiu."

- A **valid abstraction** means that the simplified steps still lead to the goal in the real world.
- A **useful abstraction** means that the steps are simple enough to follow without extra planning.
- The abstract plan Arad->Sibiu->Rimnicu Vilcea->Pitesti->Bucharest can be followed in real life along with extra details like Music or weather.
- Abstraction lets intelligent agents **ignore irrelevant details** and still solve complex problems.